U25AI055

DSA Lab-11

1. **Representation of binary tree**

```
#include <iostream>
using namespace std;
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
struct Node *buildBinaryTree()
{
   int data;
   cout << "Enter data : ";
   cin >> data;
   if (data == -1)
   {
      return NULL;
   }
   struct Node *root = new Node();
   root->data = data;
   cout << "Enter left child of " << data << " : \n";
   root->left = buildBinaryTree();
   cout << "Enter right child of " << data << " : \n";
   root->right = buildBinaryTree();
   return root;
}
void inorder(struct Node *root)
{
   if (root != NULL)
   {
      inorder(root->left);
      cout << root->data << ",";
      inorder(root->right);
   }
}
void preorder(struct Node *root)
{
   if (root != NULL)
```

```
    {
        cout << root->data << ",";
        preorder(root->left);
        preorder(root->right);
    }
}
void postorder(struct Node *root)
{
    if (root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << ",";
    }
}
int main()
{
    struct Node *root = buildBinaryTree();
    cout << "Inorder : ";
    inorder(root);
    cout << endl;
    cout << "Preorder : ";
    preorder(root);
    cout << endl;
    cout << "Postorder : ";
    postorder(root);
    cout << endl;
    return 0;
}
```

Output:

```
Enter data : 5
Enter left child of 5 :
Enter data : 3
Enter left child of 3 :
Enter data : 1
Enter left child of 1 :
Enter data : -1
Enter right child of 1 :
Enter data : 2
Enter left child of 2 :
Enter data : -1
Enter right child of 2 :
Enter data : -1
Enter right child of 3 :
Enter data : 4
Enter left child of 4 :
Enter data : -1
Enter right child of 4 :
Enter data : -1
Enter right child of 5 :
Enter data : 6
Enter left child of 6 :
Enter data : -1
Enter right child of 6 :
Enter data : 8
Enter left child of 8 :
Enter data : 7
Enter left child of 7 :
Enter data : -1
```

2. **Tree traversal**

```
#include <iostream>
using namespace std;
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};
struct Node *buildBinaryTree()
{
   int data;
   cout << "Enter data : ";
   cin >> data;
   if (data == -1)
   {
      return NULL;
   }
   struct Node *root = new Node();
   root->data = data;
```

```
    cout << "Enter left child of " << data << " : \n";
    root->left = buildBinaryTree();
    cout << "Enter right child of " << data << " : \n";
    root->right = buildBinaryTree();
    return root;
}
void inorder(struct Node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        cout << root->data << ",";
        inorder(root->right);
    }
}
void preorder(struct Node *root)
{
    if (root != NULL)
    {
        cout << root->data << ",";
        preorder(root->left);
        preorder(root->right);
    }
}
void postorder(struct Node *root)
{
    if (root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << ",";
    }
}
int main()
{
    struct Node *root = buildBinaryTree();
    cout << "Inorder : ";
    inorder(root);
    cout << endl;
    cout << "Preorder : ";
    preorder(root);
```

```
    cout << endl;
    cout << "Postorder : ";
    postorder(root);
    cout << endl;
    return 0;
}
```

Output:

```
Enter data : 5
Enter left child of 5 :
Enter data : 3
Enter left child of 3 :
Enter data : 1
Enter left child of 1 :
Enter data : -1
Enter right child of 1 :
Enter data : 2
Enter left child of 2 :
Enter data : -1
Enter right child of 2 :
Enter data : -1
Enter right child of 3 :
Enter data : 4
Enter left child of 4 :
Enter data : -1
Enter right child of 4 :
Enter data : -1
Enter right child of 5 :
Enter data : 6
Enter left child of 6 :
Enter data : -1
Enter right child of 6 :
Enter data : 8
Enter left child of 8 :
Enter data : 7
Enter left child of 7 :
Enter data : -1
```

```
Enter right child of 7 :
Enter data : -1
Enter right child of 8 :
Enter data : -1
Inorder : 1,2,3,4,5,6,7,8,
Preorder : 5,3,1,2,4,6,8,7,
Postorder : 2,1,4,3,7,8,6,5,
```

3. **BST – Insertion**

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
struct Node *insert(struct Node *root, int data)
{
    if (root == NULL)
    {
```

```
        root = new Node();
        root->data = data;
        root->left = NULL;
        root->right = NULL;
        return root;
    }
    if (data < root->data)
    {
        root->left = insert(root->left, data);
    }
    else if (data > root->data)
    {
        root->right = insert(root->right, data);
    }
    return root;
}
void inorder(struct Node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        cout << root->data << ",";
        inorder(root->right);
    }
}
int main()
{
    int n, data;
    cout << "Enter number of nodes : ";
    cin >> n;
    struct Node *root = NULL;
    for (int i = 1; i <= n; i++)
    {
        cout << "Enter data : ";
        cin >> data;
        root = insert(root, data);
    }
    inorder(root);
    return 0;
}
```

Output:

```
Enter number of nodes : 12
Enter data : 20
Enter data : 15
Enter data : 30
Enter data : 10
Enter data : 17
Enter data : 25
Enter data : 40
Enter data : 2
Enter data : 12
Enter data : 16
Enter data : 19
Enter data : 27
2,10,12,15,16,17,19,20,25,27,30,40,
```

4. **BST – Searching**

```
#include <iostream>
using namespace std;
struct Node
{
  int data;
  struct Node *left;
  struct Node *right;
};
struct Node *insert(struct Node *root, int data)
{
  if (root == NULL)
  {
    root = new Node();
    root->data = data;
    root->left = NULL;
    root->right = NULL;
    return root;
  }
  if (data < root->data)
  {
    root->left = insert(root->left, data);
  }
  else if (data > root->data)
  {
    root->right = insert(root->right, data);
  }
  return root;
}
```

```
void inorder(struct Node *root)
{
   if (root != NULL)
   {
      inorder(root->left);
      cout << root->data << ",";
      inorder(root->right);
   }
}
bool search(struct Node *root, int data)
{
   if (root == NULL)
   {
      return false;
   }
   if (root->data == data)
   {
      return true;
   }
   else if (data < root->data)
   {
      return search(root->left, data);
   }
   else
   {
      return search(root->right, data);
   }
}
int main()
{
   int n, data;
   struct Node *root = NULL;
   cout << "Enter number of nodes : ";
   cin >> n;
   for (int i = 1; i <= n; i++)
   {
      cout << "Enter data : ";
      cin >> data;
      root = insert(root, data);
   }
   inorder(root);
```

```
    cout << endl;
    cout << "Enter data to search : ";
    cin >> data;
    if (search(root, data))
    {
        cout << data << " was found\n";
    }
    else
    {
        cout << data << " was not found\n";
    }
    return 0;
}
```

Output:

```
Enter number of nodes : 12
Enter data : 20
Enter data : 15
Enter data : 30
Enter data : 10
Enter data : 17
Enter data : 25
Enter data : 40
Enter data : 2
Enter data : 12
Enter data : 16
Enter data : 19
Enter data : 27
2,10,12,15,16,17,19,20,25,27,30,40,
Enter data to search : 15
15 was found
```

5. **BST – Deletion**

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
struct Node *insert(struct Node *root, int data)
{
    if (root == NULL)
    {
        root = new Node();
```

```
        root->data = data;
        root->left = NULL;
        root->right = NULL;
        return root;
    }
    if (data < root->data)
    {
        root->left = insert(root->left, data);
    }
    else if (data > root->data)
    {
        root->right = insert(root->right, data);
    }
    return root;
}
void inorder(struct Node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        cout << root->data << ",";
        inorder(root->right);
    }
}
struct Node *findMax(struct Node *root)
{
    if (root != NULL)
    {
        if (root->right != NULL)
        {
            return findMax(root->right);
        }
        return root;
    }
}
struct Node *deleteNode(struct Node *root, int data)
{
    if (root == NULL)
    {
        return NULL;
    }
```

```
    if (data < root->data)
    {
        root->left = deleteNode(root->left, data);
    }
    else if (data > root->data)
    {
        root->right = deleteNode(root->right, data);
    }
    else
    {
        if (root->left == NULL)
        {
            return root->right;
        }
        if (root->right == NULL)
        {
            return root->left;
        }
        struct Node *temp = findMax(root->left);
        root->data = temp->data;
        root->left = deleteNode(root->left, temp->data);
    }
    return root;
}
int main()
{
    int n, data;
    struct Node *root = NULL;
    cout << "Enter number of nodes : ";
    cin >> n;
    for (int i = 1; i <= n; i++)
    {
        cout << "Enter data : ";
        cin >> data;
        root = insert(root, data);
    }
    inorder(root);
    cout << endl;
    cout << "Enter element to delete : ";
    cin >> data;
    root = deleteNode(root, data);
```

```
    inorder(root);
    cout << endl;
    return 0;
}
```

Output:

```
Enter number of nodes : 12
Enter data : 20
Enter data : 15
Enter data : 30
Enter data : 10
Enter data : 17
Enter data : 25
Enter data : 40
Enter data : 2
Enter data : 12
Enter data : 16
Enter data : 19
Enter data : 27
2,10,12,15,16,17,19,20,25,27,30,40,
Enter element to delete : 15
2,10,12,16,17,19,20,25,27,30,40,
```